B.5 Comparison with Rabe and Staats [66]

We describe here some similarities and differences between our FlashAttention algorithm and the algorithm of Rabe and Staats [66].

Conceptually, both FlashAttention and Rabe and Staats [66] operate on blocks of the attention matrix using the well-established technique of tiling (or softmax scaling) [51, 60]. To reduce the memory footprint, both methods avoid storing the large attention matrix in the forward pass and recompute it in the backward pass.

The first major difference is that Rabe and Staats [66] focuses on the reducing the total memory footprint (maximum amount of GPU memory required) while FlashAttention focuses on reducing memory accesses (the number of memory reads/writes). As mentioned in Section 2, the amount of memory access is the primary determining factor of runtime. Reducing memory accesses also necessarily reduces the total amount of memory required (e.g., if an operation incurs A memory accesses, then its total memory requirement is at most A). As a result, FlashAttention is faster than standard attention (2-4×) while Rabe and Staats [66] is around the same speed or slightly slower than standard attention. In terms of total memory required, both methods offer substantial memory saving.

The second difference between the two methods is the way information is summarized from each block to pass to the next block. Rabe and Staats [66] summarizes each block with its temporary output along with the softmax normalization statistics. At the end of the forward pass, the temporary outputs of all the blocks are combined using the statistics to produce the final output. FlashAttention instead incrementally updates the output (Algorithm 1 line 12) after processing each block, so only one copy of the output is needed (instead of K copies for K blocks). This means that FlashAttention has smaller total memory requirement compared to Rabe and Staats [66].

The final major difference is the way the backward pass is computed. Rabe and Staats [66] uses gradient checkpointing to recompute the attention matrix and the temporary output of each block. FlashAttention instead simplifies the backward pass analytically (Appendices B.2 and B.4). It only recomputes the attention matrix and does not recompute the temporary output of each block. This reduces the memory requirement for the backward pass and yields speedup.

C Proofs

Proof of Theorem 1. We first count the number of FLOPs and extra memory required.

The dominating FLOPs are from matrix multiplication. In the inner loop, (Algorithm 1 line 9), we compute $\mathbf{Q}_i\mathbf{K}_j^\top \in \mathbb{R}^{B_r\times B_c}$ for $\mathbf{Q}_i \in \mathbb{R}^{B_r\times d}$ and $\mathbf{K}_j \in \mathbb{R}^{B_c\times d}$, which takes $O(B_rB_cd)$ FLOPs. We also compute (Algorithm 1 line 12) $\tilde{\mathbf{P}}_{ij}\mathbf{V}_j \in \mathbb{R}^{B_r\times d}$ for $\tilde{\mathbf{P}}_{ij} \in \mathbb{R}^{B_r\times B_c}$ and $\mathbf{V}_j \in \mathbb{R}^{B_c\times d}$, which takes $O(B_rB_cd)$ FLOPs. We execute the inner loops $T_cT_r = \left\lceil \frac{N}{B_c}\right\rceil\left\lceil \frac{N}{B_r}\right\rceil$ times. Therefore the total number of FLOPs is

$$O\left(\frac{N^2}{B_cB_r}B_rB_cd\right) = O(N^2d).$$

In terms of extra memory required, we see that we need $O(N)$ memory to store the statistics (ℓ, m).

We now prove the algorithm's correctness by induction on j for $0 \le j \le T_c$. Let $\mathbf{K}_{:j} \in \mathbb{R}^{jB_c\times d}$ be the first jB_c rows of $\mathbf{K}$, and similarly $\mathbf{V}_{:j} \in \mathbb{R}^{jB_c\times d}$ the the first jB_c rows of $\mathbf{V}$. Let $\mathbf{S}_{:,:j} = \mathbf{Q}\mathbf{K}_{:j}^\top \in \mathbb{R}^{N\times jB_c}$, and $\mathbf{P}_{:,:j} = \mathrm{softmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^{N\times jB_c}$ (softmax applied row-wise). Let $m^j, \ell^{(j)}, \mathbf{O}^{(j)}$ be the values of $m, \ell, \mathbf{O}$ in HBM after the j-th iteration of the outer loop (Algorithm 1 line 5). (Note that these values of $m, \ell, \mathbf{O}$ are updated after each iteration of the outer loop.) We want to show that after the j-th iteration of the outer loop, we have computed in HBM:

$$m^{(j)} = \mathrm{rowmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^N, \quad \ell^{(j)} = \mathrm{rowsum}(\exp(\mathbf{S}_{:,:j} - m^{(j)})) \in \mathbb{R}^N, \quad \mathbf{O}^{(j)} = \mathbf{P}_{:,:j}\mathbf{V}_{:j} \in \mathbb{R}^{N\times d}.$$

Based on our initialization (Algorithm 1 line 2), this claim is true for $j = 0$ (i.e., before the any iteration of the outer loop is executed). Suppose that the claim holds for some $j = 0, \ldots, T_c - 1$. We want to show that the claim also holds for $j + 1$. Indeed, when we update the statistics in the inner loop (Algorithm 1 line 10)

on the $(j+1)$-th iteration of the outer loop, we update $m^{(j+1)} = \max(m^{(j)}, \tilde{m})$ where $\tilde{m} \in \mathbb{R}^N$ is the row-max of $\mathbf{S}_{:,j:j+1}$, the slice of $\mathbf{S}$ from column jB_c to column $(j+1)B_c - 1$. This implies that

$$m^{(j+1)} = \mathrm{rowmax}(\mathbf{S}_{:,:j+1}) \in \mathbb{R}^N.$$

Similarly, we update

$$\ell^{(j+1)} = e^{m^{(j)} - m^{(j+1)}} \ell^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{\ell},$$

where $\tilde{\ell} = \mathrm{rowsum}(\exp(\mathbf{S}_{:,j:j+1} - \tilde{m})) \in \mathbb{R}^N$. By the same algebraic manipulation in Section 3.1, we obtain:

$$\ell^{(j+1)} = \mathrm{rowsum}(\exp(\mathbf{S}_{:,:j+1} - m^{(j+1)})) \in \mathbb{R}^N.$$

Let $\mathbf{V}_{j:j+1}$ be the slice of $\mathbf{V}$ from column jB_c to column $(j+1)B_c - 1$, we also update:

$$\begin{aligned}
\mathbf{O}^{(j+1)} &= \mathrm{diag}(\ell^{(j+1)})^{-1}(\mathrm{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{O}^{(j)} + e^{\tilde{m} - m^{(j+1)}} \exp(\mathbf{S}_{j:j+1} - \tilde{m})\mathbf{V}_{j:j+1}) \\
&= \mathrm{diag}(\ell^{(j+1)})^{-1}(\mathrm{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{P}_{:,:j}\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \mathrm{diag}(\ell^{(j+1)})^{-1}(\mathrm{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathrm{diag}(\ell^{(j)}) \exp(\mathbf{S}_{:,:j} - m^{(j)})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \mathrm{diag}(\ell^{(j+1)})^{-1}(e^{-m^{(j+1)}} \exp(\mathbf{S}_{:,:j})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \mathrm{diag}(\ell^{(j+1)})^{-1}(\exp(\mathbf{S}_{:,:j} - m^{(j+1)})\mathbf{V}_{:j} + \exp(\mathbf{S}_{j:j+1} - m^{(j+1)})\mathbf{V}_{j:j+1}) \\
&= \mathrm{diag}(\ell^{(j+1)})^{-1}\left(\exp\left(\begin{bmatrix}\mathbf{S}_{:,:j} & \mathbf{S}_{j:j+1}\end{bmatrix} - m^{(j+1)}\right)\right)\begin{bmatrix}\mathbf{V}_{:j} \\ \mathbf{V}_{j:j+1}\end{bmatrix} \\
&= \mathrm{softmax}(\mathbf{S}_{:j+1})\mathbf{V}_{:j+1}.
\end{aligned}$$

We then see that the claim is also true for $j+1$. By induction, the claim is true for all $j = 0, \ldots, T_c$.

When $j = T_c$, we conclude that the final value of $\mathbf{O}$ in HBM is $\mathrm{softmax}(\mathbf{S})\mathbf{V} = \mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$.

□

Proof of Theorem 2. We first analyze the IO complexity of standard attention implementation. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ is written to HBM.

In the first step of computing the matrix multiply $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, the inputs $\mathbf{Q}, \mathbf{K}$ are read from HBM and the output $\mathbf{S} \in \mathbb{R}^{N \times N}$ is written to HBM (Algorithm 0 line 1). This incurs $\Theta(Nd + N^2)$ HBM accesses.

In the second step of computing $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, the input $\mathbf{S}$ is read from HBM and the output $\mathbf{P}$ is written to HBM (Algorithm 0 line 2). This incurs $\Theta(N^2)$ HBM accesses.

In the last step of computing $\mathbf{O} = \mathbf{P}\mathbf{V}$, the inputs $\mathbf{P}, \mathbf{V}$ are read from global memory and the output $\mathbf{O}$ is written to HBM (Algorithm 0 line 3). This incurs $\Theta(Nd + N^2)$ HBM accesses.

Overall, standard attention implementation requires $\Theta(Nd + N^2)$ global memory accesses.

We now analyze the IO complexity of streaming attention.

Following Algorithm 1, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once (Algorithm 1 line 6). We make T_c passes over $\mathbf{Q}$ and $\mathbf{O}$, each pass loading all of $\mathbf{Q}$ and all of $\mathbf{O}$ to HBM (Algorithm 1 line 8). Therefore the number of HBM accesses is $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$.

We derive the conditions on the block sizes B_c and B_r. We need the blocks $\mathbf{K}_j$ and $\mathbf{V}_j$ of size $B_c \times d$ to fit into on-chip memory, which translates to:

$$B_c d = O(M) \Leftrightarrow B_c = O\left(\frac{M}{d}\right).$$

Similarly, we need the blocks $\mathbf{Q}_i, \mathbf{O}_i$ of size $B_r \times d$ to fit into on-chip memory, which translates to:

$$B_r d = O(M) \Leftrightarrow B_r = O\left(\frac{M}{d}\right).$$

Finally, we need the block $\mathbf{S}_{ij}$ of size $B_r \times B_c$ to fit into on-chip memory, which translates to:

$$B_r B_c = O(M).$$

We therefore set:

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, \frac{M}{B_c}\right)\right) = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

We then have:

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

As a result, the number of HBM accesses is:

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

Proof of Proposition 3. For contradiction, suppose that there exists an algorithm that computes exact attention where the number for HBM access for all $M \in [d, Nd]$ is

$$o\left(\frac{N^2d^2}{M}\right).$$

In the regime of $M = \Theta(Nd)$, this results in the number of HBM accesses:

$$o\left(\frac{N^2d^2}{Nd}\right) = o(Nd).$$

However, the input to attention (matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}$) and the output $\mathbf{O}$ have size Nd and they start out being in HBM, so if the algorithm computes exact attention it must incur at least $\Omega(Nd)$ HBM accesses. This is a contradiction. □

Proof of Theorem 5. The IO complexity of the attention backward is very similar to the IO complexity of the attention forward (Theorem 2). Here we provide a sketch of the proof.

We first analyze the IO complexity of standard attention backward pass. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the outputs $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ are written to HBM.

At each step of the standard attention backward pass, one needs to load inputs of size Nd or N^2 from HBM, and needs to write the outputs of size N^2 or Nd to HBM. This incurs $\Theta(Nd + N^2)$ HBM accesses.

We now analyze the IO complexity of FlashAttention backward pass.

Similar to Theorem 2, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once. Each element of $\mathbf{dK}$ and $\mathbf{dV}$ is only written to HBM once. We make T_c passes over $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$, each pass loading all of $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$ to HBM. We also make T_c passes over $\mathbf{dQ}$, each pass reading/writing all of $\mathbf{dQ}$ from/to HBM. Therefore the number of HBM accesses is $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$.

As in the proof of Theorem 2, the constraints on the block sizes are that:

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

We then have:

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

As a result, the number of HBM accesses is:

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□